

Módulos técnicos — Dexie, TanStack Query e shadcn/ui

Estado local, estado remoto e componentes de interface

DOJO-K · 10 de setembro de 2026

Módulos técnicos — Dexie, TanStack Query e shadcn/ui

Rastreabilidade: [issue #11](#) — [Dojo D2](#), Técnico e demonstrações orientadas.

O que aprender

- Dexie mantém dados locais e uma fila que sobrevive à recarga.
- TanStack Query organiza os dados que vêm da API.
- shadcn/ui acrescenta componentes como código-fonte da aplicação.
- As três ferramentas resolvem fronteiras diferentes e não substituem umas às outras.

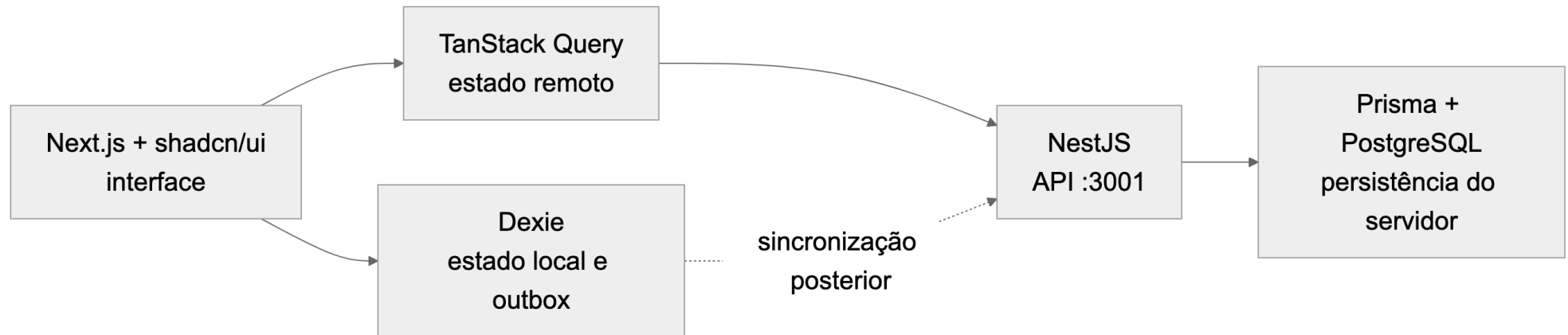


Figura 1 — cada ferramenta cuida de uma fronteira. O exercício não fecha o modelo de domínio do produto.

Dexie

DX-01 · Problema e limite

[DOCUMENTADO] Dexie é uma camada sobre IndexedDB. Neste exercício, ele mantém uma fila local que sobrevive a recarga da página. Ele não substitui o banco do servidor, não decide conflitos e não torna a aplicação offline sozinho.

Fonte: [Dexie — documentação](#), acesso em 10/09/2026.

DX-02 · Quatro conceitos

Conceito	Frase de bolso
Dexie	declara o banco local e suas tabelas
.version(n).stores()	declara índices por versão; versão anterior não é reescrita
transação	mantém juntas as mudanças locais que precisam ser atômicas
useLiveQuery	repete a consulta quando os dados observados mudam

DX-03 · Outbox mínima e indexável

```
TS
import Dexie, { type EntityTable } from 'dexie';

type EstadoEnvio = 'pendente' | 'enviado';

export interface PresencaPendente {
  id?: number;
  clientMutationId: string;
  atividadeId: string;
  pessoaId: string;
  status: 'presente' | 'ausente';
  criadoEm: number;
  estado: EstadoEnvio;
}

export const db = new Dexie('cybersetor_dojo') as Dexie & {
  presencasPendentes: EntityTable<PresencaPendente, 'id'>;
};

db.version(1).stores({
  presencasPendentes: '++id,&clientMutationId,estado,criadoEm',
});
```

[DOCUMENTADO] Booleanos não são tipos indexáveis no IndexedDB/Dexie. Por isso o exemplo usa as strings pendente e enviado, e não um índice booleano.

Fontes: [Dexie — Indexable Type](#), [Version.stores\(\)](#), acesso em 10/09/2026.

DX-04 · Gravar, consultar e observar

```
TSX
'use client';

import { useLiveQuery } from 'dexie-react-hooks';
import { db, type PresencaPendente } from './db';

export async function registrarPresenca(
  dado: Omit<PresencaPendente, 'id' | 'estado'>,
) {
  await db.presencasPendentes.add({ ...dado, estado: 'pendente' });
}

export function ContadorPendencias() {
  const pendentes = useLiveQuery(
    () => db.presencasPendentes.where('estado').equals('pendente').toArray(),
    [],
    [],
  );

  return <p>Pendentes: {pendentes.length}</p>;
}
```

DX-05 · Exercício e verificação

1. gravar duas presenças com identificadores diferentes;
2. recarregar e confirmar que as duas continuam no IndexedDB;
3. alterar uma para estado: 'enviado';
4. observar a contagem cair de 2 para 1 sem recarregar.

Passa: persistência entre recargas e atualização reativa. **Falha:** dado desaparece, índice não aceita a consulta ou a tela exige F5.

DX-06 · Decisões que ficam abertas

[PROPOSTA] O exemplo não define payload de produção, política de conflito, retentativa, exclusão da fila ou aceitação de evidência sem meta. `clientMutationId` ensina idempotência; o contrato final depende de spike e decisão da equipe.

TanStack Query

TQ-01 · Problema e limite

TanStack Query organiza **estado remoto**: carregamento, erro, cache, mutação e atualização de consultas. Dexie cuida da persistência local. Uma ferramenta não substitui a outra.

TQ-02 · Três conceitos e duas pré-condições

Peça	Papel
QueryClientProvider	disponibiliza um único cliente para os componentes
useQuery	lê e mantém em cache dados do servidor
useMutation	escreve e reage ao resultado
response.ok	separa sucesso HTTP de 400/500
'use client'	permite hooks no App Router do Next.js

[DOCUMENTADO] `fetch()` não rejeita automaticamente a Promise quando a resposta é 400 ou 500; o código precisa testar `response.ok`.

Fontes: [MDN — fetch\(\)](#), [TanStack Query — SSR e Next.js](#), [Next.js — Server and Client Components](#), acesso em 10/09/2026.

TQ-03 · Consulta e mutação coerentes com a API

```
TSX
'use client';

import { useMutation, useQuery, useQueryClient } from '@tanstack/react-query';

const API = process.env.NEXT_PUBLIC_API_URL ?? 'http://localhost:3001';

async function api<T>(path: string, init?: RequestInit): Promise<T> {
  const response = await fetch(`${API}${path}`, init);
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  return response.json() as Promise<T>;
}

export function useRequisitosDojo() {
  return useQuery({
    queryKey: ['requisitos-dojo'],
    queryFn: () => api('/requisitos-dojo'),
  });
}

export function useCriarRequisitoDojo() {
```

```
const queryClient = useQueryClient();
return useMutation({
  mutationFn: (dto: CreateRequisitoDojoDto) =>
    api('/requisitos-dojo', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(dto),
    }),
  onSuccess: () =>
    queryClient.invalidateQueries({ queryKey: ['requisitos-dojo'] }),
});
}
```

TQ-04 · Exercício e verificação

1. envolver a aplicação no QueryClientProvider;
2. listar com GET /requisitos-dojo;
3. criar com POST /requisitos-dojo;
4. confirmar a nova leitura após invalidateQueries;
5. enviar um DTO inválido e confirmar que a mutação entra em erro.

Passa: novo item aparece sem F5 e 400 não é tratado como sucesso. **Falha:** falta provider, a rota de leitura não existe ou a resposta inválida cai em onSuccess.

O código completo está em DOJO-L.

shadcn/ui

SH-01 · O que é

[DOCUMENTADO] A CLI adiciona o código dos componentes ao projeto. A equipe passa a possuir e editar esses arquivos; isso é diferente de apenas consumir uma caixa-preta em `node_modules`.

Fonte: [shadcn/ui — CLI](#), acesso em 10/09/2026.

SH-02 · Operação mínima

Dentro da aplicação Next.js **já autorizada no repositório único**:

```
BASH
npx shadcn@4.21.0 init --defaults --no-monorepo
npx shadcn@4.21.0 add button input --yes
```

O fluxo validado configura o projeto e os tokens/variáveis de estilo sem criar monorepo. O código integrado e compilável está em `web/src/`; a aplicação executável é montada por `web/montar-demo.sh`, em pasta ignorada pelo Git. Exige internet e Node 24.21.0 no momento de montar.

SH-03 · Exercício e verificação

1. localizar `components/ui/button.tsx` e `components/ui/input.tsx`;
2. abrir os arquivos e mostrar que são código editável;
3. renderizar um formulário mínimo;
4. alterar um detalhe local e conferir o resultado.

Passa: componentes existem como fonte da aplicação e renderizam. **Falha:** foi criado outro repositório/projeto, ou o grupo só encontrou um pacote em `node_modules`.

Não crie projeto à parte para esta tarefa. Se a aplicação Next.js autorizada ainda não existir, faça leitura guiada do boilerplate e deixe a execução para o follow-up individual.

O boilerplate completo e os exercícios de continuidade estão em `L-estudo-boilerplates.md`.